

Product Requirements Document (PRD): Contextualize.ai

Project Name	Contextualize.ai
Version	1.0 (MVP)
Status	Draft
Type	AI Productivity / Knowledge Management
Target Audience	Developers, Researchers, "Tab Hoarders"

1. Executive Summary

Contextualize.ai is an AI-powered "Second Brain" that solves digital hoarding. Instead of saving bookmarks into a void, users save content to a vector database. They can then chat with their library to retrieve insights, find connections between disparate articles, and generate new content based on their reading history.

Core Value Prop: "Stop saving. Start using. Chat with the internet you've already read."

2. User Personas

A. The "Vibe Coder" / Solo Dev

- Pain Point:** Saves 10 tutorials on "Next.js Auth" but forgets which one had the specific code snippet they need.
- Goal:** Instant retrieval of technical implementation details without Googling again.

B. The Content Creator

- Pain Point:** Needs to write a newsletter but has "writer's block" despite having saved 50 relevant articles.
- Goal:** "Synthesize these 5 articles into a draft for a LinkedIn post."

3. Functional Requirements (MVP)

3.1. Ingestion (Browser Extension & Direct Input)

- FR-01:** User must be able to save the current active tab via a browser extension popup.
- FR-02:** System must scrape the DOM (removing ads/nav) to extract core text (using @mozilla/readability).
- FR-03:** System must auto-generate a title, 1-sentence summary, and 3 tags using a small LLM (GPT-4o-mini).

- **FR-04:** Text must be chunked and embedded (Vectorized) for storage.

3.2. Storage & Memory

- **FR-05:** Store User Profile and Document Metadata in PostgreSQL.
- **FR-06:** Store Vector Embeddings in `pgvector` (Supabase).
- **FR-07:** Establish a strict RLS (Row Level Security) policy so users can only query their own vectors.

3.3. Retrieval & Chat (The "Brain")

- **FR-08:** Chat interface allowing natural language queries.
- **FR-09: RAG Pipeline:**
 1. Vectorize user query.
 2. Fetch top 3-5 closest semantic matches from DB.
 3. Pass matches + query to LLM (Context Window).
- **FR-10:** Responses must cite specific sources (e.g., "Based on [Article Title]...").

3.4. Visualization

- **FR-11:** "Brain Graph" visualization showing documents as nodes, clustered by semantic similarity.

4. Technical Architecture

Tech Stack

- **Frontend:** Next.js 14+ (App Router), Tailwind CSS, Lucide React (Icons).
- **Backend/DB:** Supabase (Auth, Postgres, pgvector).
- **AI Orchestration:** Vercel AI SDK (Standardizes API calls).
- **LLM Provider:** OpenAI (GPT-4o for chat, text-embedding-3-small for vectors).
- **Extension:** Plasmo Framework (React-based extension builder).

Data Schema (Simplified)

Table: documents

- `id` (uuid)
- `user_id` (uuid, FK)
- `url` (string)
- `title` (string)
- `content_raw` (text)
- `summary` (text)

- created_at (timestamp)

Table: document_embeddings

- id (uuid)
- document_id (uuid, FK)
- embedding (vector(1536))
- chunk_content (text)

5. MVP Scope vs. Roadmap

Feature	MVP (Now)	V2 (Later)
Input	Text/HTML URLs only	PDF Uploads, Youtube Transcripts
Org	Auto-tags (AI)	Manual Folders, Bi-directional linking
Chat	Q&A with Citations	"Write a Blog Post" Agent
Platform	Web + Chrome Ext	Mobile App
Visuals	Basic 2D Graph	3D Interactive WebGL Universe

6. Success Metrics (KPIs)

1. **Ingestion Rate:** Avg # of articles saved per user per week (Target: >5).
2. **Query Rate:** % of sessions where a user asks a question (Target: >40%).
3. **"Aha" Moment:** User clicks a citation link to reopen a forgotten tab.